

**MY_EPOCH 하이퍼 파라미터를
0으로 바꾸고 실험을 반복하세요.**

결과가 어떻게 바뀌나요?

(1) 문제 분석

- MY_EPOCH는 기계 학습시 학습 데이터를 몇 번을 반복하며 학습할지를 지정
- 0으로 세팅한다는 것은 학습을 한 번도 안한다는 의도
- 가중치와 편향 값을 보정하지 않고 임의의 초기값 그대로 사용한다는 의미

(2) 예상 답

- 정확도: 감소 예상
- 학습 시간: 0으로 예상

1) 실습문제 1/7

MY_EPOCH 하이퍼 파라미터를 0으로 바꾸고 실험을 반복하세요. 결과가 어떻게 바뀌나요?

(1) 정답

- 정확도: 결과는 87.5%에서 15.3%로 대폭 감소
- 혼동행렬: 대각선 외의 위치에 숫자 증가
- 학습시간: 41.7초에서 0.8초로 감소

(2) 추가문제

- MY_EPOCH을 20으로 늘려보세요.

최적화 함수를 Adam에서
SGD (경사하강법)으로 바꾸어 보세요.
결과가 어떻게 바뀌나요?

(1) 문제분석

- Adam은 경사하강법을 개선한 최적화 알고리즘
- Step 방향지정을 더욱 스마트하게 개선
- Step 크기 또한 더욱 스마트하게 개선
- Adam은 경사하강법에 비해 계산량 증가

(2) 예상답

- 정확도: 감소예상
- 학습시간: 약간감소 예상

**최적화 함수를 Adam에서 SGD (경사하강법) 으로 바꾸어
보세요. 결과가 어떻게 바뀌나요?**

(1) 정답

- 정확도: 결과는 88.0%에서 75.3%로 감소
- 혼동 행렬: 대각선 외의 위치에 숫자 증가
- 학습시간: 41.9초에서 41.3초로 감소

(2) 추가문제

- Adam 대신 RMSprop 최적화 함수를 사용해 보세요.

3) 실습문제 3/7

손실 함수를

`categorical_crossentropy`에서

MSE로 바꾸어 보세요.

결과가 어떻게 바뀌나요?

3) 실습문제 3/7

(1) 문제 분석

- 우리가 푸는 문제는 대상이 10개 중의 하나인 분류화 문제
- `Categorical crossentropy`는 대상이 두 개 이상인 분류화 문제에 사용
- MSE는 숫자를 맞추는 회귀 문제에 사용

(2) 예상 답

- 정확도: 감소 예상
- 학습 시간: 영향이 없음 예상

3) 실습문제 3/7

손실 함수를 Categorical_crossentropy에서 MSE로 바꾸어 보세요. 결과가 어떻게 바뀌나요?

(1) 정답

- 정확도: 결과는 87.5%에서 88.7%로 약간 증가!
- 혼동 행렬: 거의 변화 없음
- 학습 시간: 거의 변함 없음

(2) 추가 문제

- crossentropy를 poisson으로 바꾸어 보세요.

Convolution 필터의 크기를

(5, 5)로 바꾸어 보세요.

결과가 어떻게 바뀌나요?

(1) 문제분석

- Convolution에서 필터, 즉 kernel의 값은 학습이 되는 가중치
- 필터 크기가 커지면, 가중치의 숫자가 증가
- 그만큼 CNN 정확도도 증가 가능
- CNN의 규모가 커지면 학습 시간도 증가

(2) 예상 답

- 정확도: 증가예상
- 학습 시간: 증가예상

**Convolution 필터의 크기를 (5, 5)로 바꾸어 보세요.
결과가 어떻게 바뀌나요?**

(1) 정답

- 가중치 숫자: 411K에서 454K로 증가
- 정확도: 결과는 88.3%에서 88.8%로 증가
- 학습시간: 41.4초에서 76.8초로 증가

(2) 추가문제

- Convolution 필터의 크기를 (1, 1)로 바꾸어 보세요.

모든 Convolution의

채널 수를 4배로 줄여 보세요.

결과가 어떻게 바뀌나요?

(1) 문제 분석

- Convolution에서 출력 채널의 숫자는 사용되는 필터의 숫자와 동일
- 채널이 감소하면 필터의 숫자도 따라서 감소
- 학습이 되는 가중치도 감소하여, 그만큼 기계학습의 정확도도 감소 가능
- CNN의 규모가 작아지면 학습 시간도 감소

(2) 예상 답

- 정확도: 감소 예상
- 학습 시간: 감소 예상

**모든 Convolution의 채널 수를 4배로 줄여 보세요.
결과가 어떻게 바뀌나요?**

(1) 정답

- 가중치 숫자: 411K에서 102K로 감소
- 정확도: 결과는 88.0%에서 86.7%로 감소
- 학습시간: 42.0초에서 19.1초로 2배 감소

(2) 추가문제

- 모든 Convolution의 채널 수를 4배로 늘려 보세요.

모든 ReLU 활성화 함수를

sigmoid로 바꾸어 보세요.

결과가 어떻게 바뀌나요?

(1) 문제분석

- ReLU 함수의 기울기는 음수 부분에서는 0, 양수에서는 1로 고정
- Sigmoid 함수의 기울기는 음수 부분에서 0에 수렴, 양수 부분에서도 0에 수렴
- Sigmoid는 기울기 유실 문제에 취약
- ReLU는 기울기 유실 문제를 개선한 활성화 함수

(2) 예상답

- 정확도: 감소 예상
- 학습 시간: 영향 없음 예상

3) 실습문제 6/7

**모든 ReLU 활성화 함수를 sigmoid로 바꾸어 보세요.
결과가 어떻게 바뀌나요?**

(1) 정답

- 정확도: 결과는 87.6%에서 79.9%로 감소
- 혼동 행렬: 대각선 외의 위치에 숫자 증가
- 학습시간: 거의 영향없음

(2) 추가문제

- 모든 ReLU 활성화 함수를 tanh로 바꾸어 보세요.

**학습용 데이터의
전반부만 사용해 보세요.
결과가 어떻게 바뀌나요?**

(1) 문제 분석

- 기계학습에 있어 데이터의 양과 질이 기계학습의 성패를 크게 좌우함
- 학습용 데이터 6만개 중 반을 삭제하면 그만큼 정확도에 나쁜 영향을 미침
- 다만 학습 시간은 그에 비례하여 감소
- Numpy의 indexing 방법 사용

(2) 예상 답

- 정확도: 감소 예상
- 학습 시간: 감소 예상

**학습용 데이터의 전반부만 사용해 보세요.
결과가 어떻게 바뀌나요?**

(1) 정답

- 데이터 모양: 60,000에서 30,000으로 감소
- 정확도: 결과는 88.7%에서 86.4%로 감소
- 학습 시간: 41.2초에서 21.7초로 2배 감소

(2) 추가문제

- 학습용 데이터의 후반부만 사용해 보세요.